

2. Requirement Analysis

FitBuddy — Functional, non-functional, data, and interface requirements

Functional Requirements

- FR-01: Display a form for name, user ID, age, weight, fitness goal, workout intensity, and experience.
- FR-02: Validate user input using Pydantic constraints and accepted literal values.
- FR-03: Save or update a user profile in the database.
- FR-04: Generate a seven-day workout plan from the supplied profile.
- FR-05: Generate a nutrition/recovery tip based on the goal and weight.
- FR-06: Persist the generated plan.
- FR-07: Accept feedback for an existing user and revise the latest plan.
- FR-08: Provide an admin/coach page showing users and plans.
- FR-09: Allow deletion of a user and their stored plans.
- FR-10: Provide JSON API endpoints for generation, feedback, and user listing.
- FR-11: Provide a health endpoint reporting application status and whether Gemini is configured.

Non-Functional Requirements

- Usability: responsive Jinja2 interface with clear forms, result cards, and feedback controls.
- Maintainability: separate modules for database, CRUD, schemas, routes, and AI generation.
- Reliability: local fallback allows UI/database testing without a live Gemini key.
- Validation: age, weight, text lengths, intensity, experience, and feedback length are constrained.
- Performance: SQLite is sufficient for a lightweight academic/demo deployment; production scaling would require additional infrastructure.
- Security baseline: API keys are expected through environment configuration rather than hard-coded source.

Data Requirements

The User entity stores identity and fitness-profile attributes. The Plan entity stores the original generated plan, optional updated plan, nutrition tip, last feedback, and timestamps.

External Dependencies

- FastAPI and Uvicorn for the web application.
- Jinja2 and python-multipart for HTML templates and form handling.
- SQLAlchemy for persistence.
- Pydantic for request validation.
- google-genai for Gemini integration.

- httpx and pytest for testing/integration support.

Quick Reference

Requirement Area	Implementation Evidence
Input validation	app/schemas.py — UserInput and FeedbackRequest
Persistence	app/database.py, app/models.py, app/crud.py
Web routes	app/routes.py
AI generation	app/gemini_generator.py, gemini_flash_generator.py, updated_plan.py
UI	templates/*.html and static/css/style.css
Testing	tests/test_api.py

Project basis: This document is derived from the supplied FitBuddy source package, including its README, FastAPI routes, SQLAlchemy models, Jinja2 templates, CSS, and tests.